

1. Variáveis, tipos de dados, entrada e saída de dados, operadores e template string;

▼ Variáveis

Em Python, usamos variáveis para armazenar informações. É como dar nomes a caixinhas onde guardamos coisas. Por exemplo, podemos ter uma variável chamada `nome` que guarda o nome "Maria". Outras variáveis podem guardar números, como a `idade`, ou até mesmo coisas mais complicadas, como a `altura`.

```
#Definindo variáveis
```

```
nome = "Maria"
```

```
idade = 25
```

```
altura = 1.65
```

▼ Tipos de dados

▼ Python é dinamicamente tipada

Ou seja, não é necessário declarar o tipo da variável e nem fazer casting (alterar o tipo da variável), pois tudo isso é encarregado pelo interpretador.

▼ Tipos de dados padrão do Python:

▼ Inteiro (int)

Tipo de dado usado para número que pode ser escrito sem um componente decimal, também podendo ser positivo ou negativo:

```
idade = 42
```

```
dia = 31
```

```
print(type(idade))
```

```
print(type(dia))
```

```
# <class 'int'>
# <class 'int'>
```

▼ Ponto Flutuante ou Decimal (float)

Tipo de dado usado para números decimais. Esses são os números que podem ser representados por uma fração:

```
peso = 68.8
altura = 1.74

print(type(peso))
print(type(altura))

# <class 'float'>
# <class 'float'>
```

▼ Complexo(complex)

Tipo de dado usado para representar números complexos, que são usados mais em cálculos geométricos e científicos:

```
a = 12+8j
b = 6+90j

print(type(a))
print(type(b))

# <class 'complex'>
# <class 'complex'>
```

▼ String (str)

Tipo de dado que é um conjunto de caracteres que são utilizados para representar palavras, frases ou textos:

```
nome = 'Guilherme'
frase = "ele disse: 'hoje não haverá aula! e foi embora."
lugar = "mc donald's"
```

```
sobrenome = 'Henrique'

print(type(nome))
print(type(sobrenome))

# <class 'str'>
# <class 'str'>
```

▼ Boolean (bool)

Tipo de dado lógico que pode representar apenas dois valores: falso ou verdadeiro (False ou True). Pela lógica computacional eles podem ser considerados como 0 ou 1:

```
checked = True == 1
not_checked = False == 0

print(type(checked))
print(type(not_checked))

# <class 'bool'>
# <class 'bool'>
```

▼ Listas (list) *(iremos falar depois)*

As listas em Python agrupam um conjunto de elementos variados, nelas podem estar contidos: inteiros, floats, strings e até outras listas e outros tipos.

Para definirmos uma lista utilizamos colchetes para delimitar a lista e vírgulas para separar os elementos:

```
alunos = ['Ana', 'Bruna', 'Davi']
notas = [8.5, 9, 7.5]

print(type(alunos))
```

```
print(type(notas))
```

```
# <class 'list'>
```

```
# <class 'list'>
```

▼ Dicionários (dict) *(iremos falar depois)*

Os dicionários são utilizados para agruparmos elementos através da estrutura de chave e valor:

```
usuarios = { 'Alice': 12, 'Mário': 24, 'Leo': 32 }
```

```
print(type(usuarios))
```

```
# <class 'dict'>
```

▼ Tuplas (tuple) *(iremos falar depois)*

As tuplas possuem algumas semelhanças das listas, mas sua forma de definição é diferente. Elas utilizam parênteses ao invés de colchetes; contudo a diferença mais importante é o fato de que as tuplas são imutáveis, ou seja, após ter sido definida, ela não pode ser modificada:

```
valores = ( 120 , 50 , 44 , 91 )
```

```
print(type(valores))
```

```
# <class 'tuple'>
```

Ao tentar modificar uma tupla, uma mensagem de erro irá ser exibida no console:

```
valores = ( 120 , 50 , 44 , 91 )
```

```
valores[0] = 80 # TypeError: 'tuple' object does not support item assignment
```

▼ Mudando o tipo da variável

Em algum momento pode ser preciso alterar o tipo de uma variável, para isso podemos realizar essa mudança das seguintes formas:

▼ Inteiros para floats

```
n = 10  
print(float(n))
```

▼ Floats em inteiros

```
m = 350.7  
print(int(m))  
# 350
```

▼ Números em strings

```
valor = 15  
print(type(str(valor)))  
  
# <class str'>
```

Essas são algumas das formas de conversão entre os tipos de dados. Para mais informações recomendo um estudo detalhado na documentação oficial do Python.

▼ Entrada e saída

```
nome = input("Digite um nome") #Entrada  
print (nome) #Saida
```

▼ Template String

Template string é outro método usado para formatar strings em Python. Em comparação com %operator, .format() e f-strings, ele tem uma sintaxe e funcionalidade (indiscutivelmente) mais simples.

▼ % operator

```
name = "Jose"
age = 18

print( "Olá, %s. Você tem %s." % (name, age))

# 'Olá, José. Você tem 18.'
```

▼ .format()

```
print('O {} {} e {}'.format('carro','amarelo','rapido')) #chaves vazias
O carro amarelo e rápido

print('O {2} {1} e {0}'.format('car','yellow','fast')) #índice
O carro amarelo e rápido

print('O {f} {y} e {c}'.format(c='carro',y='amarelo',f='rapido')) #palavras-chave
O carro amarelo e rápido
```

▼ f-strings

```
name = "José"
print(f'Olá {name}, Prazer em conhecê-lo!')
```

▼ Operadores

<https://pythonacademy.com.br/blog/operadores-aritmeticos-e-logicos-em-python>

- **Operadores Aritméticos**
- **Operadores de Comparação**
- **Operadores de Atribuição**
- **Operadores Lógicos**
- **Operadores de Identidade**
- **Operadores de Associação**